

Video 1 — Demo bộ automation

Sinh viên 23127195 · Feature demo: FR-09 Mã giảm giá · Thời lượng ~8 phút Quay face-cam trên máy tính → **không cần** `whoami` / `hostname` .

Cách dùng file này: phần trong **khung vàng** là lời thoại — đọc nguyên văn. Phần chữ thường bên ngoài là thao tác cần làm. Các khối lệnh chỉ việc copy-paste.

PHẦN A — LỆNH COPY-PASTE

Mở **3 cửa sổ PowerShell riêng biệt**. Copy nguyên khối, dán vào, Enter.

Terminal 1 — Backend (mở trước khi quay, để yên suốt buổi)

```
cd D:\Kiem_thu\HW4\HW04-TESTING
node eshop-sut/backend/database.js
cd D:\Kiem_thu\HW4\HW04-TESTING\eshop-sut\backend
node server.js
```

Đợi hiện `Server is running on http://localhost:3000` → **để yên cửa sổ này, đừng đóng**.

Terminal 2 — Frontend web (để yên suốt buổi)

```
cd D:\Kiem_thu\HW4\HW04-TESTING\eshop-sut\frontend-web
npm run dev
```

Đợi hiện `Local: http://localhost:5173/` → **để yên cửa sổ này**.

Terminal 3 — Cửa sổ sẽ xuất hiện trong video (dọn sạch trước khi REC)

```
cd D:\Kiem_thu\HW4\HW04-TESTING
Remove-Item -Recurse -Force test-results -ErrorAction SilentlyContinue
cls
```

Hai lệnh dùng KHI ĐANG QUAY

Lệnh A — chạy 3 trình duyệt (dùng ở Mục 4, chạy khoảng 2 phút)

```
node scripts/run-multibrowser.mjs tests/fr09-coupon.spec.js
```

Lệnh B — mở báo cáo HTML (dùng ở Mục 5)

```
npx playwright show-report playwright-report/fr09-coupon-chromium
```

Xem report xong nhấn **Ctrl + C** trong Terminal 3 để tắt server report, không thì nó chiếm terminal và bạn không gõ tiếp được.

Checklist trước khi bấm REC

- ☐ Terminal 1 hiện `Server is running on http://localhost:3000`
- ☐ Terminal 2 hiện `Local: http://localhost:5173/`
- ☐ Mở Chrome vào `http://localhost:5173` → thấy trang chủ EShop có sản phẩm
- ☐ **VS Code** mở sẵn 2 tab: `tests/data/fr09-coupon-calculations.csv` và `tests/fr09-coupon.spec.js`
- ☐ **Chrome** mở sẵn 2 tab: `http://localhost:5173/checkout` và `https://github.com/hungtmh/HW04-TESTING/issues`
- ☐ Phóng to cỡ chữ Terminal và VS Code (`Ctrl` + `+` vài lần)
- ☐ Bật webcam, thử mic 10 giây rồi nghe lại
- ☐ Tắt Focus Assist / Zalo / Messenger

PHẦN B — KỊCH BẢN QUAY

Mục 1 · Mở đầu — 40 giây

Hình: Face-cam toàn màn hình

Em xin chào thầy cô và các bạn. Em là Trần Minh Hùng, mã số sinh viên hai ba một hai bảy một chín năm. Đây là video báo cáo bài tập HW04, môn Kiểm thử phần mềm, phần Automation Testing.

Trong bài này em đã tự động hoá ba feature của hệ thống EShop bằng Playwright. Video hôm nay em sẽ demo chi tiết một feature là **FR-09, Mã giảm giá**.

Nội dung video gồm bốn phần. Thứ nhất, em giới thiệu cấu trúc bộ test và cách em tổ chức dữ liệu kiểm thử. Thứ hai, em chạy thật bộ test đó trên ba trình duyệt khác nhau. Thứ ba, em mở báo cáo HTML và chứng minh lỗi tìm được là lỗi thật của hệ thống. Và cuối cùng, phần em cho là quan trọng nhất, em sẽ kể lại **một lỗi mà chính em đã phát hiện và sửa** trong đoạn script do AI sinh ra.

Mục 2 · Giới thiệu hệ thống — 50 giây

Hình: Chrome → `http://localhost:5173` (lướt qua trang chủ)

Hệ thống được kiểm thử là **EShop**, một website thương mại điện tử demo được thiết kế riêng cho việc luyện tập kiểm thử. Hệ thống gồm ba phần chạy song song: backend API ở cổng ba nghìn, giao diện người mua ở cổng năm một bảy ba, và trang quản trị ở cổng năm một bảy tư.

Theo đề bài, em phải chọn đúng ba feature đã chọn ở HW02, mỗi nhóm chức năng một feature. Ba feature của em là: **FR-01 đăng ký tài khoản** thuộc nhóm A, **FR-09 mã giảm giá** thuộc nhóm B, và **FR-14 quản lý danh mục** thuộc nhóm C.

Tổng cộng em viết được **62 test case**. Mỗi feature đều chạy trên ba engine trình duyệt khác nhau, nên tổng số lượt chạy là **186 lượt**, tương ứng **9 browser run** — vượt yêu cầu tối thiểu của đề là 9.

Mục 3 · Cấu trúc bộ test — 1 phút 30

Hình: VS Code → tab `tests/data/fr09-coupon-calculations.csv`

Đề bài có một yêu cầu rất rõ ràng: bộ test phải **data-driven**, nghĩa là dữ liệu kiểm thử bắt buộc phải nằm ở file riêng, không được viết cứng trong mã nguồn. Nếu viết cứng thì bài bị loại.

Đây là file CSV chứa ma trận mười trường hợp tính giảm giá của FR-09. Mỗi cột lần lượt là: mã test case, mô tả, mã giảm giá, loại mã, tổng tiền đơn hàng, số tiền giảm mong đợi, thành tiền mong đợi, và kết quả mong đợi.

(Trò chuột vào dòng TC-01, dừng 2 giây)

Em lấy ví dụ dòng đầu tiên. Mã SAVE10 là mã giảm mười phần trăm. Áp lên đơn năm trăm nghìn thì theo toán học phải giảm năm mươi nghìn, và khách còn phải trả bốn trăm năm mươi nghìn.

Em muốn nhấn mạnh một điểm rất quan trọng: những con số này là con số **theo đặc tả**, tức là theo tài liệu API và theo đúng ý nghĩa của phép tính phần trăm. Nó **không phải** là con số mà hệ thống đang trả về. Lát nữa các thầy cô sẽ thấy vì sao điều này quyết định toàn bộ chất lượng của bộ test.

Hình: VS Code → chuyển sang tab `tests/fr09-coupon.spec.js` , cuộn chậm từ đầu file

Còn đây là file spec, tức file chứa các kịch bản kiểm thử. FR-09 có 18 test case, chia thành bảy nhóm.

Ngay đầu file em ghi chú rõ **sáu kiểu assertion** đã sử dụng, trong khi đề bài chỉ yêu cầu tối thiểu ba kiểu. Cụ thể: P1 kiểm tra điều hướng trang, P2 kiểm tra phần tử và văn bản hiển thị trên giao diện, P3 kiểm tra giá trị số, P4 gọi thẳng API của backend để đối chiếu dữ liệu thật, P5 đếm số phần tử, và P6 kiểm tra các **bất biến** — tức những điều kiện luôn luôn phải đúng với mọi mã giảm giá.

(Cuộn xuống nhóm 1, chỗ vòng lặp `for`)

Đây là chỗ đọc file CSV rồi sinh test bằng vòng lặp. Mỗi dòng trong CSV trở thành một test case riêng. Nhờ vậy, khi em muốn bổ sung trường hợp kiểm thử mới thì chỉ cần thêm một dòng vào file CSV, hoàn toàn không phải sửa mã nguồn.

(Trò vào một test có nhãn `@bug`)

Các thầy cô để ý những test có gắn nhãn `@bug`. Đây là các test em **cố ý để cho fail**. Chúng kiểm tra theo đúng đặc tả, còn hệ thống thì đang làm sai, nên kết quả tất yếu là đỏ.

Em không hề nói lỏng chúng cho xanh, bởi vì nếu sửa test cho khớp với hành vi sai của hệ thống thì chẳng khác nào biến lỗi thành đặc tả. Mỗi test đỏ ở đây tương ứng chính xác với một lỗi thật đã được em ghi vào bug report và đăng lên GitHub Issues.

Mục 4 · Chạy thật trên 3 trình duyệt — 2 phút ⚠ BẮT BUỘC

Hình: Terminal 3 — dán **Lệnh A**

```
node scripts/run-multibrowser.mjs tests/fr09-coupon.spec.js
```

Nói liên tục trong lúc chờ (khoảng 2 phút):

Em vừa chạy lệnh này. Script sẽ lần lượt chạy file spec trên Chromium, rồi Firefox, rồi WebKit. Mỗi engine sẽ sinh ra **một báo cáo HTML riêng biệt**.

Có thể thầy cô thắc mắc tại sao em phải viết script riêng thay vì gõ thẳng `npx playwright test`. Lý do là nếu chạy mặc định, Playwright sẽ gộp kết quả của cả ba trình duyệt vào **một báo cáo duy nhất**. Trong khi đề bài yêu cầu rõ là mỗi browser run phải sinh ra một HTML report. Nên script này đặt biến môi trường khác nhau cho từng lần chạy để tách thư mục báo cáo ra.

(Khi Chromium chạy xong, có số liệu)

Chromium đã xong: **12 test pass, 6 test fail**. Sáu test fail này chính là các test gắn nhãn `@bug` mà em vừa trình bày.

(Trong lúc Firefox đang chạy)

Trong lúc chờ, em xin nói thêm về ba trình duyệt này. Đây là ba engine khác nhau **về bản chất**, chứ không phải ba vỏ bọc của cùng một engine. Chromium dùng nhân Blink, giống Chrome và Edge. Firefox dùng nhân Gecko. Còn WebKit chính là nhân của trình duyệt Safari trên máy Mac và iPhone.

Playwright tải sẵn cả ba trình duyệt này về máy, dung lượng khoảng năm trăm megabyte, đặt trong thư mục AppData. Đây là **trình duyệt thật, đầy đủ chức năng**, không phải giả lập.

Cách Playwright điều khiển chúng là mở một kênh WebSocket hai chiều tới tiến trình trình duyệt, rồi gửi lệnh qua đó: mở tab mới, truy cập địa chỉ này, tìm phần tử khớp với điều kiện kia, click chuột tại toạ độ đó, gõ chuỗi ký tự này. Với Chromium thì giao thức đó tên là Chrome DevTools Protocol — chính là thứ mà tab DevTools khi ta bấm F12 đang sử dụng.

Vì là click chuột thật tại toạ độ thật, nên ứng dụng React bên trong xử lý y hệt như khi người dùng tự thao tác, kích hoạt đúng các sự kiện thật.

(Khi bảng tổng kết ba dòng hiện ra)

Và đây là bảng tổng kết. Cả ba engine đều cho kết quả **hoàn toàn giống nhau: 12 pass, 6 fail**.

Sự đồng nhất này rất quan trọng, vì nó chứng tỏ bộ test **không bị flaky** — không có test nào lúc xanh lúc đỏ tùy trình duyệt hay tùy tốc độ máy. Để đạt được điều này em đã phải sửa một số lỗi trong chính bộ test của mình, mà lát nữa em sẽ kể.

Mục 5 · Mở báo cáo HTML — 1 phút ⚠ BẮT BUỘC

Hình: Terminal 3 — dán **Lệnh B**

```
npx playwright show-report playwright-report/fr09-coupon-chromium
```

(Trình duyệt tự mở. Trỏ chuột vào dòng tiêu đề, dừng lại 3 giây)

Em mở báo cáo HTML của lần chạy trên Chromium.

Xin thầy cô đặc biệt chú ý dòng tiêu đề ở đầu trang: **HW04 EShop Automation — Run by: 23127195**, kèm theo **dấu thời gian chuẩn ISO** của đúng lần chạy vừa rồi.

Đây chính là yêu cầu chống gian lận trong đề bài. Mã số sinh viên và timestamp được nhúng vào báo cáo **ngay tại thời điểm chạy**, thông qua cấu hình reporter trong file `playwright.config.js`, chứ không phải sửa tay thêm vào sau. Nếu em chạy lại lúc này thì timestamp sẽ khác đi.

(Trở vào 3 con số ở góc trên: All 18 / Passed 12 / Failed 6)

Ở đây có thống kê tổng: 18 test, 12 pass, 6 fail.

(Click vào test TC-01 đang fail)

Em click vào test TC-01 đang bị đỏ. Thông báo lỗi ghi rất rõ: **kỳ vọng số tiền giảm là 50 nghìn, nhưng thực tế nhận được âm 4 triệu 500 nghìn.**

Playwright còn lưu lại cả trace, tức bản ghi từng thao tác, và ảnh chụp màn hình tại thời điểm test thất bại.

Mục 6 · Chứng minh lỗi là thật — 1 phút

Hình: Chuyển sang Chrome tab `http://localhost:5173/checkout`

Bây giờ em sẽ chứng minh đây là lỗi thật của hệ thống, chứ không phải do em viết test sai. Em sẽ thao tác hoàn toàn bằng tay trên giao diện.

(Điền ô Tổng tiền thanh toán = `500000`)

Em đặt tổng tiền thanh toán là năm trăm nghìn đồng.

(Nhập mã `SAVE10` vào ô Mã Giảm Giá, bấm nút Áp dụng)

Em nhập mã SAVE10 — là mã giảm mười phần trăm theo dữ liệu gốc của hệ thống — rồi bấm nút Áp dụng.

(Dừng lại, trở chuột vào từng dòng kết quả)

Và đây là kết quả. Giao diện hiện dấu tích màu xanh, kèm dòng chữ "Áp dụng thành công, giảm mười phần trăm". Nhìn qua thì tưởng mọi thứ bình thường.

Nhưng nhìn xuống hai dòng dưới: **Tiết kiệm âm bốn triệu năm trăm nghìn đồng.** Và **Thành tiền: năm triệu đồng.**

Nghĩa là đơn hàng năm trăm nghìn, sau khi áp mã giảm giá, khách hàng phải trả năm triệu — **gấp đúng mười lần.**

Nguyên nhân nằm ở backend. Công thức được viết là: tổng tiền nhân với, mở ngoặc, một trừ giá trị giảm giá. Với mã mười phần trăm thì giá trị giảm giá lưu trong database là số 10, nên biểu thức thành một trừ mười, bằng **âm chín**. Số tiền giảm trở thành số âm. Mà thành tiền được tính bằng tổng tiền trừ đi số tiền giảm, trừ một số âm thì thành cộng, nên kết quả là mười lần tổng đơn hàng.

Công thức đúng phải là: tổng tiền nhân phần trăm rồi chia cho một trăm.

Đây là lỗi nghiêm trọng nhất em tìm được trong toàn bộ bài, em đã ghi thành **Issue số 7** trên GitHub, kèm ảnh chụp và các bước tái hiện.

Mục 7 · Lỗi em đã sửa trong script do AI sinh — 1 phút 40 ⚠ BẮT BUỘC

Đây là phần đề bài bắt buộc phải có. Nói chậm, rõ, đừng vội.

Hình: Face-cam, hoặc VS Code mở `tests/fr01-register.spec.js` phần chú thích R-03

Phần cuối cùng, em xin kể về một lỗi mà chính em đã phát hiện và sửa trong đoạn code do AI sinh ra.

Em chọn kể lỗi này vì nó là lỗi **nguy hiểm nhất** trong tất cả những lỗi em gặp. Lý do: nó **không làm test báo đỏ**. Nó làm test **báo xanh giả**.

Bối cảnh là ở feature FR-01, đăng ký tài khoản. Em cần kiểm tra rằng khi người dùng nhập email sai định dạng, ví dụ chỉ gõ chữ "a b c", thì hệ thống phải từ chối và không được tạo tài khoản.

AI viết đoạn kiểm tra như sau: sau khi bấm nút Đăng ký, nó kiểm tra xem địa chỉ URL trên thanh trình duyệt có còn nằm ở trang `/register` hay không. Lập luận là: nếu đăng ký thành công thì hệ thống sẽ chuyển sang trang đăng nhập; vậy nếu vẫn còn ở trang đăng ký thì tức là đã bị từ chối.

Nghe rất hợp lý. Và **bốn test viết theo cách đó đều báo xanh**.

Nhưng khi em thử lại bằng tay, em phát hiện hệ thống **thực tế vẫn chấp nhận** email "a b c" và vẫn tạo tài khoản bình thường. Tức là bốn test kia đáng lẽ phải báo đỏ mới đúng.

Em truy nguyên và hiểu ra nguyên nhân. Ứng dụng này là **single-page application**.

Ngay tại thời điểm vừa bấm nút, request gửi lên server **chưa xử lý xong**, cho nên địa chỉ URL đương nhiên vẫn còn là `/register` — bất kể server sẽ quyết định chấp nhận hay từ chối. Câu lệnh kiểm tra khớp điều kiện **ngay lập tức** và chuyển sang xanh, trước cả khi ứng dụng kịp quyết định bất cứ điều gì.

Nói cách khác: **bộ test đã che giấu đúng cái lỗi mà nó được viết ra để tìm**.

Cách em sửa là bỏ hẳn việc suy đoán qua URL. Thay vào đó em hỏi thẳng cơ sở dữ liệu thông qua API quản trị: hiện còn bao nhiêu dòng người dùng mang địa chỉ email này? Nếu hệ thống từ chối đúng như đặc tả thì con số phải là **không**.

Sau khi sửa, cả bốn test chuyển sang màu đỏ. Và đó mới là kết quả phản ánh **đúng** chất lượng thật của hệ thống. Bốn test đỏ này về sau trở thành **Issue số 2** trên GitHub.

Bài học em rút ra, và em nghĩ đây là bài học lớn nhất của cả bài tập: **một test báo xanh không chứng minh được rằng phần mềm đúng. Nó có thể chỉ chứng minh rằng câu lệnh kiểm tra đã được đặt sai chỗ**.

Mục 8 · Kết — 30 giây

Hình: Chrome tab GitHub Issues, cuộn chậm qua danh sách

Em xin tổng kết. Trong bài HW04 này em đã tự động hoá ba feature với 62 test case, thực hiện 9 browser run trên ba engine khác nhau, và phát hiện tổng cộng **15 lỗi**.

Toàn bộ 15 lỗi đều đã được đăng lên GitHub Issues, mỗi issue đều có mô tả nguyên nhân, các bước tái hiện, ảnh chụp bằng chứng và đề xuất cách sửa.

Ngoài ra em còn đóng gói toàn bộ quy trình này thành một Agent Skill tái sử dụng được, em sẽ trình bày trong video thứ hai.

Em xin chân thành cảm ơn thầy cô đã dành thời gian theo dõi.

PHẦN C — KIỂM TRA SAU KHI QUAY

- ☐ Video **dài hơn 5 phút**
- ☐ Có **mặt bạn** (face-cam) và **giọng bạn** xuyên suốt
- ☐ Có cảnh chạy **đủ 3 trình duyệt** và bảng tổng kết
- ☐ Báo cáo HTML hiện **"Run by: 23127195"** + ISO timestamp, **đọc được rõ chữ**
- ☐ Có đoạn tái hiện lỗi BUG-07 bằng tay trên giao diện
- ☐ Có đoạn kể **lỗi đã sửa** trong script AI (Mục 7)
- ☐ Upload YouTube → chọn **Unlisted** (Không công khai)
- ☐ Dán link vào `23127195/README.md`